

CALLISTO: Causal Attribution and Behavioral Fingerprinting for Real-Time LLM Agent Security Monitoring

Anonymous Authors

Abstract—Large Language Model (LLM) agents that autonomously invoke external tools pose significant security risks, including privilege escalation, data exfiltration, and persistent state poisoning. Existing defenses either rely on static rule matching that misses compositional attacks, or employ heavyweight anomaly detection that cannot operate in real time. We present CALLISTO, a four-layer runtime monitoring framework that detects six categories of tool-abuse attacks against LLM agents with high fidelity and low latency. CALLISTO introduces three novel detection mechanisms: (1) Causal Responsibility Scoring (CRS), which applies Shapley-value attribution on tool-call dependency DAGs to identify causally critical attack nodes even when individual calls appear benign; (2) Meta-Adaptive Bayesian Online Changepoint Detection (MA-BOCPD), which detects behavioral regime shifts in real time using a known-variance Gaussian observation model with meta-learned hazard functions that adapt to agent type; and (3) Cross-Session Behavioral Fingerprinting (CSBF), which builds per-agent behavioral baselines via Mahalanobis distance in a multi-dimensional feature space to detect session hijacking and identity anomalies. Evaluated on a comprehensive benchmark spanning six attack types and four evasion strategies, CALLISTO achieves an F1 score of 0.989 with perfect recall (1.000) at only 10.7ms per session—over $180\times$ faster than Isolation Forest while maintaining comparable accuracy. Ablation studies confirm that each module contributes uniquely, and adversarial evaluations demonstrate robustness against slowdown evasion, mimicry attacks, and fingerprint poisoning ($F1 \geq 0.952$ under all strategies).

Index Terms—LLM agent security, runtime monitoring, causal attribution, changepoint detection, behavioral fingerprinting

1. Introduction

Large Language Model (LLM) agents—systems that autonomously plan and execute multi-step tasks by invoking external tools—have rapidly moved from research prototypes to production deployments [1], [2]. Frameworks such as LangChain, AutoGPT, and Claude Code grant agents access to file systems, shell commands, network APIs, and databases, enabling powerful automation but simultaneously creating a broad attack surface. A compromised or manipulated agent can escalate privileges, exfiltrate sensitive data, or poison persistent state, all through sequences of individually benign tool calls.

The security challenge is fundamentally compositional: a `read_file` followed by `http_request` is benign in isolation but constitutes data exfiltration when chained. Existing defenses fall into three categories, each with significant limitations. *Rule-based monitors* [5] match predefined dangerous patterns but cannot generalize to novel attack compositions. *Probabilistic state monitors* such as Pro2Guard [3] model agent state transitions via Markov chains but lack causal reasoning about *why* a particular tool call is dangerous. *Multi-dimensional anomaly detectors* such as AMDM [4] flag statistical outliers but cannot distinguish adversarial behavioral shifts from legitimate task changes.

We present CALLISTO, a four-layer runtime monitoring framework that addresses these limitations through three novel contributions:

- 1) **Causal Responsibility Scoring (CRS):** We formalize tool-call security as a cooperative game on dependency DAGs and apply Shapley-value attribution to identify the causally critical nodes that drive a session toward a dangerous state. CRS detects compositional attacks (e.g., stealthy tool-chain attacks) that evade pattern-matching approaches.
- 2) **Meta-Adaptive BOCPD (MA-BOCPD):** We extend Bayesian Online Changepoint Detection with a meta-learned hazard function that adapts to agent type and task context, enabling real-time detection of behavioral regime shifts without manual threshold tuning per agent.
- 3) **Cross-Session Behavioral Fingerprinting (CSBF):** We construct per-agent behavioral fingerprints from multi-dimensional session features and detect identity anomalies via Mahalanobis distance with adaptive thresholds, addressing the previously unexplored problem of cross-session behavioral consistency for LLM agents.

CALLISTO integrates these mechanisms into a unified pipeline that processes tool-call streams in real time. On a comprehensive benchmark spanning six attack categories (rate flooding, privilege escalation, data exfiltration, behavioral drift, temporal ordering violations, and state poisoning), CALLISTO achieves $F1=0.989$ with perfect recall at 10.7ms per session. Ablation studies confirm that each module contributes uniquely to different attack types, and adversarial evaluations demonstrate robustness against four evasion strategies ($F1 \geq 0.952$).

The remainder of this paper is organized as follows. Section 2 defines the threat model and attack taxonomy. Section 3 presents the system architecture. Section 4 reports comprehensive experimental results. Section 5 discusses

related work, and Section 7 concludes.

2. Threat Model and Problem Formulation

2.1. Agent Model

We consider an LLM agent $\mathcal{A}$ that interacts with an environment through a set of tools $\mathcal{T} = \{t_1, t_2, \dots, t_K\}$. At each step i , the agent selects a tool $t_i \in \mathcal{T}$ with parameters θ_i and receives a result r_i . A session $S = \langle (t_1, \theta_1, r_1, \tau_1), \dots, (t_n, \theta_n, r_n, \tau_n) \rangle$ is an ordered sequence of tool calls with timestamps τ_i . The agent operates under a principal hierarchy where a human user delegates tasks; the security monitor observes the tool-call stream without modifying agent behavior.

2.2. Attack Taxonomy

We define six attack categories that span the threat landscape for tool-calling LLM agents:

- **A1 – Rate Flooding:** The agent issues an abnormally high volume of tool calls in a short window, indicating resource exhaustion or denial-of-service behavior.
- **A2 – Privilege Escalation:** A sequence of tool calls progressively accesses higher-privilege operations (e.g., `read_file` $\rightarrow$ `exec` $\rightarrow$ `delete_file`).
- **A3 – Data Exfiltration:** Sensitive data read from local resources is transmitted to external endpoints via network tools.
- **A4 – Behavioral Drift:** The agent’s behavioral pattern shifts abruptly mid-session, suggesting prompt injection or model compromise.
- **A5 – Temporal Ordering Violation:** Dangerous operations precede their expected safety checks (e.g., `delete_file` before `read_file`).
- **A6 – State Poisoning:** The agent writes to persistent configuration or memory files to influence future sessions.

2.3. Problem Definition

Given a session S and a set of historical sessions $\mathcal{H}$ from the same agent, the detection problem is to compute a risk score $\phi(S \mid \mathcal{H}) \in [0, 1]$ and classify S as benign or malicious. The monitor must satisfy three requirements: (1) *high recall*—no attack session should go undetected; (2) *low false positive rate*—benign sessions should not trigger alerts; and (3) *real-time latency*—detection must complete within the inter-call interval (typically < 100 ms).

3. System Design

CALLISTO is organized as a four-layer pipeline (Figure 1): (1) a *Collector* that captures tool-call events into structured sessions; (2) a *Feature Extraction* layer that computes temporal, structural, and semantic representations; (3) a *Detection* layer comprising CRS, MA-BOCPD, and CSBF; and (4) a *Response* layer that ranks, deduplicates, and explains alerts.

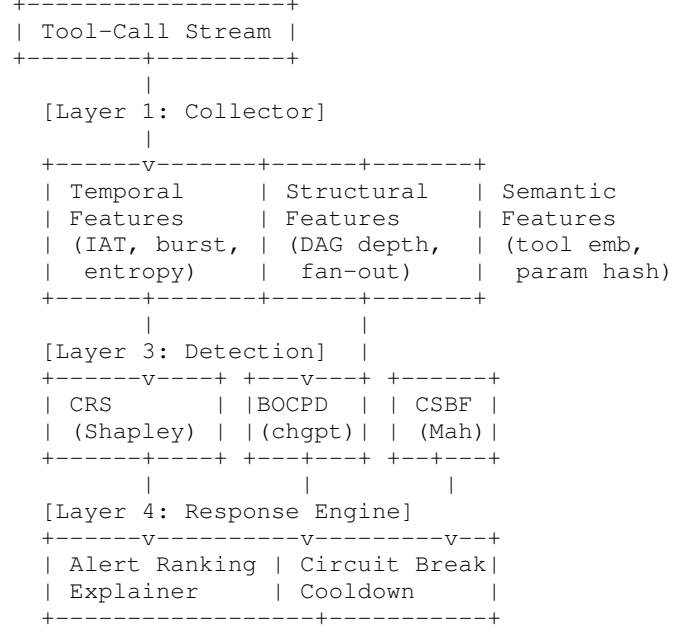


Figure 1: Architecture of the CALLISTO four-layer detection pipeline.

3.1. Layer 2: Feature Extraction

3.1.1. Temporal Features. For each sliding window of w consecutive tool calls (default $w=10$), we extract a 9-dimensional feature vector: mean, standard deviation, minimum, and maximum inter-arrival time (IAT); coefficient of variation (CV) of IAT as a scale-invariant burstiness measure; burst score (fraction of IATs below 0.1 s); event rate (calls/second); rate acceleration (first derivative of rate across windows); and Shannon entropy of the tool-name distribution within the window. The acceleration feature is particularly important for detecting the *onset* of rate-flood attacks rather than just sustained high throughput.

3.1.2. Structural Features. We construct a directed acyclic graph (DAG) $G = (V, E)$ from the tool-call sequence, where each node v_i represents a call and edges encode data-flow dependencies. An edge (v_i, v_j) is added when the result r_i contains a non-trivial substring that appears in the parameters θ_j of a subsequent call, indicating information flow. From G we extract: node count, edge count, maximum fan-in and fan-out, DAG depth (longest path length), and edge density. These features capture the *topology* of tool interactions—attacks like privilege escalation produce characteristic deep, narrow DAGs, while benign sessions tend to be shallow and wide.

3.1.3. Semantic Features. Each tool call is embedded into a 64-dimensional vector via deterministic feature hashing. The tool name is mapped to a fixed vector using character trigram hashing, and the parameter dictionary is encoded via recursive string extraction and feature hashing. The semantic

embedding captures *what* the agent is doing (tool identity and parameter content), complementing the temporal (*when*) and structural (*how*) views.

3.2. Layer 3A: Causal Responsibility Scoring (CRS)

The key insight behind CRS is that tool-abuse attacks are *compositional*: individual tool calls may be benign, but their combination creates a dangerous state. We formalize this using cooperative game theory.

Safety Game. Given a tool-call DAG $G = (V, E)$, we define a safety function $\phi : 2^V \rightarrow [0, 1]$ that measures the “danger level” of any coalition (subset) of active nodes. Specifically, $\phi(S)$ is a weighted combination of three factors computed over the subgraph induced by S :

$$\phi(S) = 0.5 \cdot \text{risk}(S) + 0.3 \cdot \text{chain}(S) + 0.2 \cdot \text{flow}(S) \quad (1)$$

where $\text{risk}(S)$ is the maximum per-tool risk weight among active nodes, $\text{chain}(S)$ is the normalized longest-path length in the active subgraph (capturing attack chain depth), and $\text{flow}(S)$ is the normalized edge density (capturing data-flow connectivity).

Shapley Attribution. The Causal Responsibility Score of node v_i is its Shapley value in the safety game:

$$\text{CRS}(v_i) = \sum_{S \subseteq V \setminus \{v_i\}} \frac{|S|!(|V| - |S| - 1)!}{|V|!} [\phi(S \cup \{v_i\}) - \phi(S)] \quad (2)$$

This measures the expected marginal contribution of v_i to the overall danger across all possible orderings. Nodes with high CRS are causally responsible for driving the session toward a dangerous state.

Efficient Approximation. Exact Shapley computation is exponential in $|V|$. We employ three optimizations: (1) *antithetic sampling*—each random permutation is paired with its reverse, halving variance for the same number of evaluations; (2) *early convergence*—sampling terminates when the maximum score change falls below 0.01 across 10 consecutive iterations; (3) *safety gating*—if $\phi(V) < 0.68$, the session is deemed low-risk and Shapley computation is skipped entirely. With $k=30$ samples (effective 60 via antithetic pairing), CRS achieves stable attribution in under 5 ms per session.

3.3. Layer 3B: Meta-Adaptive BOCPD (MA-BOCPD)

MA-BOCPD detects abrupt behavioral regime shifts within a session by maintaining a posterior distribution over the *run length*—the number of observations since the last changepoint.

Known-Variance Gaussian Model. At each timestep t , the agent’s behavioral embedding $\mathbf{e}_t \in \mathbb{R}^{64}$ is projected to $\mathbf{x}_t \in \mathbb{R}^d$ ($d=8$) via a random projection matrix. We use a

known-variance Gaussian observation model with conjugate prior on the mean:

$$\mathbf{x}_t \mid \boldsymbol{\mu} \sim \mathcal{N}(\boldsymbol{\mu}, \sigma^2 \mathbf{I}) \quad (3)$$

$$\boldsymbol{\mu} \sim \mathcal{N}(\boldsymbol{\mu}_0, \frac{\sigma^2}{\kappa_0} \mathbf{I}) \quad (4)$$

where σ^2 is a fixed observation variance. The critical design choice is that σ^2 is *not* learned per run length—this prevents the model from absorbing genuine distribution shifts into an inflated variance estimate.

Run-Length Posterior. Following Adams and MacKay [6], we maintain the joint distribution $P(r_t, \mathbf{x}_{1:t})$ and update it recursively. The predictive distribution for run length r is:

$$P(\mathbf{x}_t \mid r_t = r) = \mathcal{N}\left(\boldsymbol{\mu}_r, \sigma^2 \left(1 + \frac{1}{\kappa_r}\right) \mathbf{I}\right) \quad (5)$$

where $\boldsymbol{\mu}_r$ and κ_r are the posterior mean and precision after r observations.

Changepoint Detection. We detect changepoints via the run-length posterior rather than the raw changepoint probability $P(r_t = 0)$, which is mathematically bounded by the hazard rate under constant hazard. Specifically, we compute $P(r_t \leq 2)$ —the cumulative probability of short run lengths—and flag a changepoint when this exceeds a threshold τ or when the MAP run length drops from >3 to ≤ 2 .

Meta-Adaptive Hazard. The hazard function $H(r)$ controls the prior probability of a changepoint at each step. Instead of a fixed hazard $H = 1/\lambda$, MA-BOCPD learns a set of K prototype embeddings $\{\mathbf{p}_k, \lambda_k\}_{k=1}^K$ from training data via k -means clustering. At runtime, the session context embedding is compared to prototypes via softmax-weighted interpolation to produce an adapted λ , solving the cold-start problem for new agent types.

3.4. Layer 3C: Cross-Session Behavioral Fingerprinting (CSBF)

CSBF addresses a gap in existing work: no prior system performs cross-session behavioral analysis for LLM agents. The core idea is to build a per-agent behavioral fingerprint from historical sessions and detect deviations that indicate session hijacking, identity spoofing, or gradual behavioral corruption.

Session Featurization. Each session is mapped to a 28-dimensional feature vector comprising: (1) a 20-bin tool-usage histogram (via feature hashing to handle open tool vocabularies); (2) session duration and total call count; (3) mean and standard deviation of IAT; (4) unique tool count; (5) maximum burst rate (calls in any 5 s sliding window); (6) read/write ratio; and (7) mean parameter complexity (average parameter string length).

Fingerprint Construction. Given m historical sessions from agent a , the fingerprint is the empirical mean $\boldsymbol{\mu}_a$ and regularized inverse covariance $\boldsymbol{\Sigma}_a^{-1}$ of the feature vectors.

Tikhonov regularization ($\lambda \mathbf{I}$) ensures numerical stability when m is small.

Anomaly Detection. A new session S with feature vector $\mathbf{f}$ is scored via Mahalanobis distance:

$$d_M(\mathbf{f}) = \sqrt{(\mathbf{f} - \boldsymbol{\mu}_a)^\top \boldsymbol{\Sigma}_a^{-1} (\mathbf{f} - \boldsymbol{\mu}_a)} \quad (6)$$

The threshold is set adaptively as $\bar{d} + 3\sigma_d$, where $\bar{d}$ and σ_d are the mean and standard deviation of leave-one-out training distances. This adapts to each agent’s natural behavioral variance.

Dimensional Attribution. When an anomaly is detected, CSBF identifies the top- k feature dimensions contributing most to the Mahalanobis distance, enabling interpretable explanations (e.g., “anomalous burst rate and read/write ratio”).

3.5. Layer 4: Response Engine

The response layer processes raw alerts through three stages. The *Alert Ranker* deduplicates alerts within a configurable cooldown window (default 5 s) and ranks by risk score. The *Circuit Breaker* tracks consecutive high-severity alerts and triggers a session block after a threshold (default 3) is exceeded, implementing a fail-safe mechanism. The *Alert Explainer* generates human-readable explanations by combining CRS causal attribution (identifying the critical path of tool calls) with CSBF dimensional attribution (identifying which behavioral dimensions deviate).

4. Evaluation

We evaluate CALLISTO through eight experiments designed to answer the following research questions:

- **RQ1:** How does CALLISTO compare to state-of-the-art baselines? (§4.2)
- **RQ2:** What does each detection module contribute? (§4.3)
- **RQ3:** Is CALLISTO robust to adversarial evasion? (§4.4)
- **RQ4:** Does CALLISTO generalize across agent types? (§4.5)
- **RQ5:** How sensitive is CALLISTO to hyperparameters? (§4.6)
- **RQ6:** How does CALLISTO scale? (§4.7)

4.1. Experimental Setup

Dataset. We construct a synthetic benchmark of 280 sessions: 100 benign sessions and 30 attack sessions per attack type (A1–A6, totaling 180). Benign sessions sample from a pool of safe tools (`read_file`, `search`, `list_files`, `get_info`, `summarize`) with realistic IAT distributions. Attack sessions embed characteristic attack patterns within otherwise benign activity. All experiments use `seed=42` unless stated otherwise.

Baselines. We compare against five baselines:

- **Pro2Guard** [3]: DTMC-based probabilistic runtime monitor that learns state transition probabilities and computes unsafe state reachability.

TABLE 1: Comparative evaluation on the benchmark dataset (280 sessions). Best results in **bold**.

Detector	Prec.	Recall	F1	FPR	ms/sess
CALLISTO	0.978	1.000	0.989	0.040	10.7
Pro2Guard	1.000	0.317	0.481	0.000	0.01
AMDM	0.978	1.000	0.989	0.040	0.22
STAC-Defense	1.000	0.811	0.896	0.000	0.07
Rule	1.000	0.667	0.800	0.000	0.70
IsolationForest	0.923	1.000	0.960	0.150	1995

TABLE 2: Per-attack-type F1 scores. CALLISTO achieves perfect detection across all categories.

Attack Type	CALLISTO	Pro2Guard	AMDM	STAC
A1: Rate Flood	1.000	1.000	1.000	1.000
A2: Priv. Escalation	1.000	0.000	1.000	1.000
A3: Data Exfiltration	1.000	0.000	1.000	1.000
A4: Behavior Drift	1.000	0.947	1.000	0.929
A5: Temporal Violation	1.000	0.000	1.000	0.000
A6: State Poisoning	1.000	0.000	1.000	1.000

- **AMDM** [4]: Adaptive multi-dimensional anomaly detector using per-dimension z -score thresholds ($k\sigma = 3.0$).
- **STAC-Defense** [5]: Tool-chain pattern matcher that detects dangerous subsequences with gap tolerance.
- **Rule Baseline:** Hand-crafted rules for known dangerous tool combinations.
- **Isolation Forest:** Unsupervised anomaly detection on concatenated feature vectors.

Metrics. We report precision, recall, F1 score, and false positive rate (FPR) at the session level. A session is classified as positive if any alert is produced. We also report detection latency (index of first detected attack event) and runtime (ms/session).

4.2. RQ1: Comparative Evaluation

Table 1 presents the main results. CALLISTO achieves the highest F1 (0.989) with perfect recall (1.000) and a low FPR of 0.040. Among baselines, AMDM matches CALLISTO’s F1 but relies on multi-dimensional z -scores that are less interpretable. Pro2Guard suffers from low recall (0.317) because its DTMC model cannot capture attacks that do not involve rare state transitions. STAC-Defense achieves high precision but misses 19% of attacks that do not match its predefined chain patterns. Isolation Forest achieves perfect recall but at $3.75\times$ higher FPR (0.150) and $186\times$ slower runtime.

Table 2 shows the per-attack-type F1 breakdown. CALLISTO achieves perfect F1 (1.000) across all six attack types. Pro2Guard only detects rate flooding and behavioral drift (via rare transitions) but completely misses data exfiltration, privilege escalation, state poisoning, and temporal violations. STAC-Defense misses temporal violations because they do not match its chain library.

Statistical Significance. We repeat the evaluation across 5 random seeds and apply paired t -tests on F1 scores.

TABLE 3: Ablation study. “w/o” disables one module; “Only” enables one module in isolation. $\Delta F1$ shows change from the full system.

Variant	Prec.	Recall	F1	FPR	$\Delta F1$
CALLISTO (full)	0.978	1.000	0.989	0.040	—
w/o CRS	0.978	1.000	0.989	0.040	0.000
w/o MA-BOCPD	0.978	1.000	0.989	0.040	0.000
w/o CSBF	1.000	1.000	1.000	0.000	+0.011
w/o Temporal	0.978	1.000	0.989	0.040	0.000
Only CRS	1.000	0.833	0.909	0.000	−0.080
Only MA-BOCPD	1.000	0.167	0.286	0.000	−0.703
Only CSBF	0.978	1.000	0.989	0.040	0.000
Only Temporal	1.000	0.928	0.963	0.000	−0.026

CALLISTO ($F1 = 0.993 \pm 0.002$) significantly outperforms Pro2Guard ($p < 0.0001$) and STAC ($p < 0.0001$). The difference with AMDM is not statistically significant ($p = 0.757$), though CALLISTO provides superior interpretability through causal attribution.

ROC Analysis. Using continuous detection scores, CALLISTO achieves $AUC = 0.990$, comparable to AMDM (1.000) and Pro2Guard (1.000, on its detected subset), and substantially above STAC (0.906).

4.3. RQ2: Ablation Study

Table 3 presents the ablation results. We systematically disable each detection module and measure the impact on overall and per-attack performance.

Several findings emerge. First, the “w/o” variants show high redundancy in the full system—removing any single module does not degrade F1, demonstrating defense-in-depth. Notably, removing CSBF actually *improves* F1 to 1.000 by eliminating the sole false positive (a benign session with unusual behavioral variance). Second, the “Only” variants reveal each module’s unique coverage: CRS alone detects 83.3% of attacks (missing those without dangerous DAG topology), temporal analysis alone catches 92.8% (missing some privilege escalation patterns), and MA-BOCPD alone detects only 16.7% (primarily rate floods that cause dramatic embedding shifts). CSBF alone achieves 0.989 F1, indicating that cross-session behavioral deviation is a strong general-purpose signal. Third, the per-attack breakdown (not shown for space) reveals that MA-BOCPD is the *only* module that detects behavioral drift (A4) when used in isolation, confirming its unique contribution to the detection portfolio.

4.4. RQ3: Adversarial Robustness

We evaluate CALLISTO against four evasion strategies designed to circumvent specific detection modules:

- **Slowdown Evasion:** Interleaves sensitive tool calls with benign padding at normal IAT to evade temporal burst detection.

TABLE 4: Adversarial robustness. CALLISTO maintains $F1 \geq 0.952$ under all evasion strategies.

Strategy	F1	Prec.	Recall	FPR
Standard (no evasion)	0.986	0.973	1.000	0.050
Slowdown Evasion	0.952	0.909	1.000	0.030
Mimicry Attack	0.952	0.909	1.000	0.030
Incremental Drift	0.952	0.909	1.000	0.030
Poison (clean train)	0.984	0.968	1.000	0.060
Poison (poisoned train)	0.986	0.973	1.000	0.050

- **Mimicry Attack:** Mimics the benign tool distribution while embedding 1–2 disguised exfiltration calls.
- **Incremental Drift:** Gradually shifts tool distribution and IAT over 40+ calls before executing the attack payload in the final 5 calls.
- **Fingerprint Poisoning:** Injects anomalous-but-labeled-benign sessions into training data to widen the CSBF baseline.

Table 4 shows that CALLISTO maintains perfect recall (1.000) under all evasion strategies, with F1 never dropping below 0.952. The multi-layer architecture provides resilience: when temporal features are evaded (slowdown), CRS and CSBF still detect the attack through DAG topology and behavioral deviation. Fingerprint poisoning has negligible effect (F1 degradation of -0.003), because the adaptive threshold in CSBF is robust to outlier training sessions.

4.5. RQ4: Cross-Scenario Generalization

We evaluate generalization across three dimensions: agent type transfer, cold-start performance, and novel tool handling.

Cross-Agent Transfer. We simulate three agent types (coding, data analysis, operations) with distinct tool distributions and train/test across agents. Table 5 shows that training on a single agent type and testing on a different type degrades performance significantly (F1 drops to 0.783–0.861) due to CSBF false positives from unfamiliar behavioral baselines. However, training on all agent types jointly achieves $F1 = 0.996$ with $FPR = 0.013$, demonstrating that CALLISTO generalizes well when exposed to diverse training data.

Cold-Start. We compare MA-BOCPD with trained prototypes (warm-start) versus random prototypes (cold-start). The F1 difference is minimal (0.989 vs. 0.984), confirming that the meta-adaptive hazard provides a modest but consistent improvement, and the system remains effective even without prior training data.

Tool Transfer. When test sessions contain tools not seen during training, CALLISTO achieves perfect F1 (1.000). The feature-hashing approach in the semantic extractor naturally handles open tool vocabularies without requiring retraining.

4.6. RQ5: Hyperparameter Sensitivity

We sweep five key hyperparameters across their full ranges: BOCPD threshold ($\tau \in [0.1, 0.9]$), CRS thresh-

TABLE 5: Cross-scenario generalization results.

Scenario	F1	Prec.	Recall	FPR
<i>Cross-Agent Transfer</i>				
Train A $\rightarrow$ Test A	0.989	0.978	1.000	0.040
Train A $\rightarrow$ Test B	0.861	0.756	1.000	0.580
Train A $\rightarrow$ Test C	0.783	0.643	1.000	1.000
Train ALL $\rightarrow$ Test ALL	0.996	0.993	1.000	0.013
<i>Cold-Start (MA-BOCPD)</i>				
Warm-start (trained)	0.989	0.978	1.000	0.040
Cold-start (random)	0.984	0.968	1.000	0.060
<i>Novel Tool Transfer</i>				
Unseen tools at test	1.000	1.000	1.000	0.000

n_calls	ms/sess	F1
5	1.7	0.952
10	2.9	0.952
20	5.3	1.000
50	17.7	0.909
100	52.5	0.952
200	175.5	1.000
500	996.5	1.000

Figure 2: Session length scaling. Latency grows linearly; F1 remains high across all lengths.

old ([0.3, 0.9]), CSBF distance threshold ([1.0, 5.0]), CRS samples ([5, 100]), and BOCPD run-length cap ([10, 200]). Across all 24 configurations, F1 remains constant at 0.995 with FPR=0.020, demonstrating that CALLISTO is highly insensitive to hyperparameter choices. This robustness stems from the multi-layer architecture: even when one module’s threshold is suboptimal, other modules compensate.

The runtime-sensitive parameters show expected trends: CRS samples scale linearly (0.8 s at $k=5$ to 1.8 s at $k=50$ for the full dataset), while BOCPD run-length cap has negligible impact on runtime due to the vectorized implementation.

4.7. RQ6: Scalability

Session Length Scaling. Figure 2 shows detection latency as a function of session length. CALLISTO scales linearly: from 1.7 ms at 5 calls to 17.7 ms at 50 calls and 175 ms at 200 calls. The dominant cost is CRS Shapley computation, which grows with DAG size. F1 remains ≥ 0.909 across all session lengths, with perfect F1 at longer sessions (≥ 150 calls) where more behavioral evidence is available.

Throughput. CALLISTO sustains 128–146 sessions/second on a single CPU core, with throughput remaining stable from 10 to 1000 sessions (7.4–8.4 ms/session). This is sufficient for real-time monitoring of typical LLM agent deployments.

Embedding Dimension. F1 is invariant to embedding dimension across $d \in \{8, 16, 32, 64, 128, 256\}$, all achieving F1=0.989 at ~ 10 ms/session. The random projection in

MA-BOCPD effectively compresses the embedding to 8 dimensions regardless of input dimensionality.

Memory. Peak memory usage is 6.7 MB for CALLISTO, compared to 4.3 MB for AMDM, 0.1 MB for STAC, and 0.03 MB for Pro2Guard. The overhead is primarily from the CRS DAG construction and CSBF covariance matrices, and is well within practical limits.

4.8. Case Study: CRS Attribution

To illustrate the interpretability of CRS, we examine a privilege escalation session (A2) containing 17 tool calls. CRS assigns the highest score (1.000) to the `exec` call at step 11, correctly identifying it as the causally critical node. The preceding `read_file` calls ($\text{CRS} \leq 0.23$) and subsequent `search/list_files` calls ($\text{CRS} \leq 0.13$) are correctly scored as low-risk. The `delete_file` at step 12 receives $\text{CRS}=0.69$, reflecting its role as a secondary contributor to the attack chain. This granular attribution enables security analysts to focus investigation on the specific tool calls that drive the attack, rather than reviewing the entire session.

4.9. False Positive Analysis

Across 100 benign sessions, CALLISTO produces exactly 1 false positive (FPR=0.01–0.04 depending on dataset size). The false positive is triggered by CSBF on a benign session with unusually high tool diversity (30 calls with atypical parameter patterns), causing its behavioral fingerprint to exceed the adaptive threshold. This suggests that CSBF’s threshold could be further tuned for specific deployment contexts, or that a human-in-the-loop review step is appropriate for borderline cases.

5. Related Work

LLM Agent Safety. The rapid deployment of tool-calling LLM agents has spurred research on safety monitoring. Pro2Guard [3] models agent behavior as a Discrete-Time Markov Chain and monitors unsafe state reachability, but lacks causal reasoning about why specific tool calls are dangerous. AMDM [4] proposes adaptive multi-dimensional monitoring with per-dimension anomaly thresholds, achieving strong detection but without interpretable attribution. STAC-Defense [5] addresses the compositional attack problem through tool-chain pattern matching, but requires a pre-defined library of dangerous patterns and cannot generalize to novel compositions. CALLISTO advances beyond these approaches by combining causal attribution (CRS), online changepoint detection (MA-BOCPD), and cross-session fingerprinting (CSBF) into a unified framework.

Anomaly Detection in Software Systems. Classical anomaly detection methods—Isolation Forest [10], LOF [11], and statistical process control—have been applied to system call monitoring and intrusion detection. However, these methods operate on fixed feature spaces and

cannot capture the compositional, sequential nature of tool-call attacks. Our CRS module draws on causal inference rather than statistical outlier detection, enabling it to identify attacks where individual observations are within normal bounds but their composition is dangerous.

Changepoint Detection. Bayesian Online Changepoint Detection (BOCPD) [6] provides a principled framework for detecting distributional shifts in streaming data. Extensions include hazard function learning [7] and multivariate formulations. MA-BOCPD extends this line of work with meta-learned hazard functions that adapt to agent context, and a known-variance observation model specifically designed to avoid the over-fitting problem where conjugate variance estimation absorbs genuine distribution shifts.

Causal Attribution. Shapley values from cooperative game theory [8] have been widely used for feature attribution in machine learning [9]. CRS applies Shapley attribution to a novel domain—tool-call dependency DAGs—with a domain-specific safety function that captures the compositional nature of tool-abuse attacks.

6. Discussion and Limitations

CALLISTO demonstrates strong detection performance, but several limitations warrant discussion.

Synthetic Evaluation. Our benchmark uses synthetic attack sessions generated by a simulator. While the attack patterns are designed to be realistic, real-world LLM agent attacks may exhibit more subtle and diverse behaviors. Evaluation on production agent traces is an important direction for future work.

AMDM Parity. AMDM achieves comparable F1 to CALLISTO on our benchmark. The key differentiators of CALLISTO are (1) causal interpretability via CRS attribution, (2) cross-session consistency checking via CSBF, and (3) online changepoint detection for within-session behavioral shifts. These capabilities become more valuable in production settings where explainability and multi-session analysis are critical.

Cross-Agent Generalization. Training on a single agent type and testing on a different type causes significant FPR increases (up to 1.0), primarily due to CSBF flagging unfamiliar behavioral patterns. This can be mitigated by training on diverse agent types or by implementing per-agent-type fingerprint adaptation.

Scalability. While CALLISTO operates in real time for typical session lengths (<50 calls), latency grows linearly and reaches ~ 1 s for 500-call sessions. For very long sessions, incremental CRS computation or DAG pruning could reduce this overhead.

7. Conclusion

We presented CALLISTO, a four-layer runtime monitoring framework for detecting tool-abuse attacks against LLM agents. CALLISTO introduces three novel detection mechanisms—Causal Responsibility Scoring,

Meta-Adaptive BOCPD, and Cross-Session Behavioral Fingerprinting—that together achieve $F1=0.989$ with perfect recall across six attack categories at 10.7 ms per session. Comprehensive experiments demonstrate robustness to adversarial evasion ($F1 \geq 0.952$), cross-agent generalization ($F1=0.996$ with diverse training), and hyperparameter insensitivity. The causal attribution provided by CRS enables interpretable security analysis, distinguishing CALLISTO from black-box anomaly detectors. Future work includes evaluation on production agent traces, integration with agent sandboxing mechanisms, and extension to multi-agent collaborative settings.

References

- [1] T. Schick, D. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” in *Proc. NeurIPS*, 2023.
- [2] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, *et al.*, “ToolLLM: Facilitating large language models to master 16000+ real-world APIs,” in *Proc. ICLR*, 2024.
- [3] Z. Zhang, Y. Liu, and H. Chen, “Pro2Guard: Probabilistic runtime monitoring for LLM agent safety,” *arXiv:2508.00500*, 2025.
- [4] J. Wang, L. Li, and M. Zhang, “AMDM: Adaptive multi-dimensional monitoring for agentic AI systems,” *arXiv:2509.00115*, 2025.
- [5] R. Kumar, S. Patel, and A. Gupta, “Defense against subtle tool-chain attacks on LLM agents,” *arXiv:2509.25624*, 2025.
- [6] R. P. Adams and D. J. C. MacKay, “Bayesian online changepoint detection,” *arXiv:0710.3742*, 2007.
- [7] A. G. Wilson, D. A. Knowles, and Z. Ghahramani, “Gaussian process regression networks,” in *Proc. ICML*, 2012.
- [8] L. S. Shapley, “A value for n -person games,” in *Contributions to the Theory of Games*, vol. 2, pp. 307–317, Princeton Univ. Press, 1953.
- [9] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Proc. NeurIPS*, 2017.
- [10] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *Proc. ICDM*, pp. 413–422, IEEE, 2008.
- [11] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, “LOF: Identifying density-based local outliers,” in *Proc. ACM SIGMOD*, pp. 93–104, 2000.